n = 19 tau6 index calculus end to end in native code, v2 (N2 against the current row's arm N): frozen protocol

2026-10-21

n = 19 tau6 index calculus end to end in native code, v2: frozen protocol

Status (2026-10-21): frozen and launched; no result yet. The protocol (`protocol.json`) was frozen at 2026-10-21T05:34:45Z, before any timing. The workflow is `n19-native-loop-v2.yml`. See Launch for the run. Nothing in this README is a timing result.

This is a re-freeze of `volcano-n19-native-loop-20261021` (v1, archived in PR #1016). v1's run 39526183111 was INVALID: its Sage smoke gate failed and the timed job never ran. The question, the candidate, the baseline, the arms, the gates and the statistics are v1's. Three things change:

1. the defect that crashed v1's planted arm and profile is fixed;
2. the checks that missed it now run before any timing;
3. the held-out panel and every seed are new.

Question

The current n = 19 row is arm N of `volcano-n19-native-tau6-v3-20261020` (PR #1005, run 39387473809). It is Tier 1, with cold CPU B / N = 141.02 [136.75, 144.95] and cold `S` = 31.42 in that run's rho unit. N is Sage's `tauDlp` running the v3 native per-shift solve through ctypes.

Does candidate **N2** lower the end-to-end cold CPU of one complete, verified ECDLP against N? N2 is the `tau6dlp` binary: the whole pipeline in native code, with no Sage process. Every target, shift order, shift system, relation, matrix, kernel and log must stay unchanged. The crate (`native/`) is v1's, byte for byte; its per-shift solve is the v3 source file, included by path.

Why v1 failed: root cause, with evidence



Figure 1: v1's failure mechanism, v2's fix, and the v2 smoke job's order. Source: make_figures.py (`argv`); vector: figures/argv.svg.

What the runner showed. In run 39526183111 both smoke N2X processes (serials 9 and 16) exited 1 with the same traceback. It is archived on the v1 branch as `run_github_39526183111/smoke/logs/smoke-009.err`:

```
File ".../arm_n2x.sage.py", line 159, in <module>
File ".../arm_n2x.sage.py", line 107, in main
    arm, out, lib, v3lib = opt('--arm'), opt('--out'), os.path.abspath(opt('--lib')), ...
TypeError: expected str, bytes or os.PathLike object, not NoneType
```

The launcher's command line for that row did carry `--lib /rd/lib/libtau6loop.so`. `diagnosis/profile_n.sage` died the same way (`profile_n.sage.py` line 313 → line 264) in diagnostic run 39522833864 and again in the smoke job.

The cause. The two scripts share one global name with v3 arm.sage. arm_n2x.sage did:

v1 line	code	`_argv` afterwards
arm_n2x.sage 36	`_argv = list(sys.argv)`	the full command line
arm_n2x.sage 37, 40	`sys.argv = [ARM3]; exec(preparse(v3 code), globals())`	
v3 arm.sage 62 (inside that exec, **same globals**)	`_argv = sys.argv`	`[v3 arm.sage]`
v3 arm.sage 63–65	`sys.argv = [run.sage]; load(run.sage); sys.argv = _argv`	`[v3 arm.sage]`
arm_n2x.sage 41	`sys.argv = _argv`	`sys.argv` is now `[v3 arm.sage]`

v1 line	code	_argv afterwards
arm_n2x.sage 100–102 (= arm_n2x.sage.py 105–107: sage adds a 5-line header)	a = sys.argv[1:] is empty, so opt('--lib') is None	abspath(None) raises TypeError

The exec(..., globals()) runs v3's module-level code in arm_n2x's own namespace, so v3's _argv = sys.argv rebinds arm_n2x's saved copy. profile_n.sage lines 34–39 have the same pattern. The task's starting hypothesis was that opt() reads sys.argv instead of _argv. That is true, but it is not the cause: arm_n2x does restore sys.argv from _argv before opt() runs. What it restores is v3's value.

The preparser is not involved. Integer literals become Integer(1) in a.index(k) + 1, and that indexes a list fine; v3's arm N parses its options the same way and ran correctly on the runner. v3 arm.sage run on its own also works, because nothing it loads (run.sage, ic.sage, tau.sage) binds _argv.

The evidence.

- tests/test_option_path.py runs **v1's own files** the way sage FILE.sage does (tests/sage_emulation.py). That means Sage's preparse_file layout (a 5-line header, hoisted _sage_const_ literals), Integer literals, ^ → **, and one __main__ namespace shared with every load. It uses no stand-in that parses options.
- The emulation **reproduces both archived tracebacks line for line**:
  - TypeError in abspath at arm_n2x.sage.py 159 → 107;
  - TypeError in abspath at profile_n.sage.py 313 → 264.
- After the v3 load, the namespace's _argv equals [.../volcano-n19-native-tau6-v3-20261020/arm.sage].
- The same test passes on v2.
- tests/namespace_audit.py lists every module-level name that v3 arm.sage, run.sage, ic.sage, tau.sage and nbsat.sage bind (91). It finds exactly one clobbered global in each v1 script, _argv, and none in v2.

Why no earlier check saw it. v1's local dry run used a stand-in sage (smoke/fake_sage.py). The stand-in read the options from its own command line and never executed arm_n2x.sage, so the option path was tested only on the runner.

The fix and the audit

The fix (minimal, in the two scripts only). arm_n2x.sage and diagnosis/profile_n.sage now run the v3 code through _exec_v3_definitions(path, globals()). That function keeps the caller's argv in a function local, which code run in the globals cannot rebind, and restores it in a finally. A missing option now exits with its name (arm_n2x.sage: missing option(s) --lib; sys.argv = [...]) instead of a TypeError. Two additions serve the check below:

- arm_n2x.sage --options-only loads the v3 definitions exactly as a real run does, then parses and echoes every option, and solves nothing;
- profile_n.sage --mode load now echoes argv.

The v3 baseline, the volcano-ic sources, the crate, n2x_compare.py and the comparison logic are unchanged.

The audit: every place where argv or option parsing could be lost.

place	how it gets its options	finding
arm_n2x.sage (N2X)	sys.argv after the v3 exec	**v1 defect** (_argv clobbered); fixed
diagnosis/profile_n.sage	sys.argv after the v3 exec	**same defect**; fixed
v3 arm.sage run directly (B, N, O, rho, selftest)	its own _argv, around load(run.sage)	safe: nothing loaded binds _argv. B/N rows passed on the runner in v1
volcano-ic/run.sage	if __name__ == '__main__' and len(sys.argv) > 1 dispatch	safe: it is loaded with sys.argv = [run.sage], so the dispatch never fires

place	how it gets its options	finding
`run.sage` `HERE`, v3 `MYDIR`, `arm_n2x` `HERE_N2X`, `profile_n` `HERE`	`dirname(abspath(sys.argv[0]))`	correct: each is computed while `sys.argv[0]` is that file. `HERE` is rebound by run.sage, but profile_n reads it only before the load (audit: not clobbered)
names arm_n2x / profile_n define after the load (`compare`, `native_trace`, `TARGET_LOG`, `REQUIRED`, `opt`, `plain_run`, ...)	—	none is read by v3 code outside v3's own `main` (audit: nothing shadowed)
preparser on option code	`Integer` literals in `a.index(k) + 1`; `^` rewriting	harmless: Sage's `Integer` has `__index__`. The emulation applies the same rewrite and parses
`run_schedule.py`, `verify_receipt.py`, `option_check.py`, `startup_probe.py`	argparse under `sage -python` (never preparsed)	safe. The launcher passes argv lists, with no shell quoting
`tau6dlp` (N2, N2aa, O2, S0b)	Rust argument parser	ran on the runner in v1 (all binary rows valid) and runs in the local dry run
v1 dry-run stand-in `smoke/fake_sage.py`	parsed options itself	**the gap**: now it first runs the real `arm_n2x.sage --options-only` through the emulation and exits 1 if that fails

Pre-timing checks: what now catches this before the timed job

Every check below runs in the untimed `sage-smoke` job, and the `timed` job needs that job to pass.

1. **Local, and in CI on the host: `tests/test_option_path.py`** (the second option in the task: a faithful preparse).
   - `arm_n2x.sage --options-only` keeps every option through the v3 load. So does the N2X command `run_schedule.py` builds for the smoke N2X runs and the first and last timed ones.
   - `profile_n.sage --mode load` keeps its options.
   - A missing option names itself.
   - The namespace audit is clean.
   - On v1's sources, both archived tracebacks are reproduced (when the v1 directory is present: after #1016 merges, or with `N19_V1_DIR`).
   - Locally all 6 pass, with v1 present.
2. **On the runner, under the pinned image's real `sage`: `option_check.py`.** This is a hard gate, run before the profile and the smoke part.
   - It runs `sage arm_n2x.sage` with the exact commands of the two smoke N2X runs and of the first and last timed N2X runs, plus `--options-only`.
   - It runs `sage diagnosis/profile_n.sage --mode load`.
   - Each must exit 0 and echo argv and every option exactly; the two libraries must exist; the v3 definitions must be loaded.
   - Otherwise it prints `OPTION-PATH CHECK FAILED: ...` with the stderr tail and the job stops.
   - Locally, with a `sage` shim running the emulation, it passes on v2 (`smoke/option_check_emulated.json`), and with v1's `arm_n2x.sage` swapped in it fails 4 of 5 launches with v1's traceback.
3. **The smoke verifier fails on any crashed process before the gates** (the first option in the task).
   - `verify_receipt.py --part smoke` first checks that every scheduled smoke process was launched, exited 0 within its timeout and wrote its row file.
   - If any did not, it prints `SMOKE FAILED BEFORE THE GATES: k of the scheduled smoke processes ...`, with each process's serial, arm, point, exit and last stderr line. It then exits 7 without evaluating a gate.
   - The same condition is a new required gate, `no_process_crashed`, in both parts: 25 smoke gates and 28

timed gates.

- The verifier's planted-corruption tests add v1's exact failure (two N2X rows exit 1 with the `TypeError` and no row file) and a clean control: 22 of 22 pass.
- The local dry run, with the real launcher, the real `tau6dlp`, the real verifier and the stand-in `sage`, passes on v2 (`smoke/dryrun_model_sage.json`; only `tier1_counters_recorded` is false, because this VM has no `/proc/schedstat`). With v1's `arm_n2x.sage` swapped in, it stops at this check with exit 7 and `serial 9 arm N2X point v3p00: exit 1; no row file | TypeError: expected str, bytes or os.PathLike object, not NoneType` (`smoke/dryrun_v1_arm_n2x_negative.json`).

4. **The untimed profile of N runs in the same smoke job**, so no separate diagnostic run is needed.
   - `profile_n.sage` runs `--mode timers` and `--mode cprofile` on v3p00 and v3p01, and `startup_probe.py` records Sage start-up.
   - It runs after the option-path check, so its option path is already gated.
   - Its content is recorded with no claim and is not a gate: a failure is an annotated warning plus `profile/status.txt`, as in v1's protocol. A profile defect therefore cannot cost the timed run.

Frozen design (v1's, with a new panel and new seeds)



Figure 2: Port scope, checks and the frozen timed order (not a result). Source: make_figures.py; vector: figures/design.svg.

- **Panel.** 24 new scalars (`inputs/panel.json`, role `n19-native-loop-v2-heldout`, seed `volcano-n19-native-loop-v2-202`. The point rng seeds are `n19-native-loop-v2:00` to `:23`, all new.
  - **Excluded:** every earlier n = 19 scalar, its negative and every Frobenius image $\pm\lambda\hat{}e$:
    * the ten archived volcano-ic scalars;
    * the tau6 v1, v2 and v3 panels (smoke points included) and tau6 v1's local smoke panel;
    * v1's 24 panel and 2 smoke scalars. v1's frozen `inputs/panel.json` is copied byte for byte to

`inputs/excluded/`, with SHA-256 `177eb695...` checked by `make_panel.py`, because v1 may not be on main yet.
  - The panel points' τ-orbits are also disjoint from each other.
  - The smoke points stay v3 p00 and p01, because run 39387473809 archived their Sage rows. They are never timed.
- **Order.** `schedule.json` holds 508 timed processes: S0, S0b, O1, O2, then, per point, in a block order shuffled with seed `volcano-n19-native-loop-v2-20261021:order:%d`:
  - 5 alternating N/N2 pairs with a seeded start;
  - 3 N2/N2aa pairs;
  - 1 N2X/N2 pair;
  - one B, one rho and one unpaired warm N2.

  The smoke part has 18 processes.
- **Primary.** R = Σ cold CPU(N) / Σ cold CPU(N2) over the 120 primary pairs. Cold CPU is measured in-process: setup plus online on the process CPU clock (`time.process_time()` in Sage, `CLOCK_PROCESS_CPUTIME_ID` in `tau6dlp`). The rule:
  - **improvement** iff the 95% lower bound is above 1;
  - **regression** iff the upper bound is below 1.
- **Why in-process is primary (v1's reasons, unchanged).**
  - It is the current row's unit: v3's primary and the ledger's cold `S`.
  - It leaves out Sage start-up, which only N pays, so it is conservative for N2.
  - Start-up belongs to the interpreter, not to the algorithm.
- **Process-inclusive secondary, pre-registered.** Child CPU N / N2 (`getrusage` of each launched process) over the same pairs with the same bootstrap. It gets its own verdict label but never decides. Launcher wall is reported beside it.
- **Interval.** A two-stage paired bootstrap (points, then pairs within each drawn point), 10,000 draws, `random.Random(2026102111)` (new; v1 used 2026102101), 95%.
- **Controls.**
  - A/A: the N2/N2aa estimate must fall inside [0.95, 1.05], over 72 pairs.
  - Planted: the N2X/N2 lower bound must be above 1, over 24 pairs.
- **Gates.** v1's 27 (S0, S0b, O1, O2, the field check in every native process, per-point identity of counts, log, relations and representatives across B, N, N2, N2aa, N2X and warm N2, the planted item-by-item comparison, and receipt integrity), plus `no_process_crashed`. Any false, missing or unknown gate makes the run invalid.
- **Tier.** Tier 1: a fresh GitHub-hosted ubuntu-24.04 runner, one pinned exposed CPU, one fresh process per run.
  - The image is `sagemath/sagemath:10.10@sha256:310dfe23...`.
  - Rust is 1.99.0.
  - CPU-only, one host class, not replayed.
- **Time guard.**
  - The smoke job takes about 15–20 min. The timed job takes about 35 min; its worst case is about 50 min.
  - A segment is skipped, and the run made invalid, if the elapsed time plus its projection would pass 175 min.
  - The timeout is 200 min.

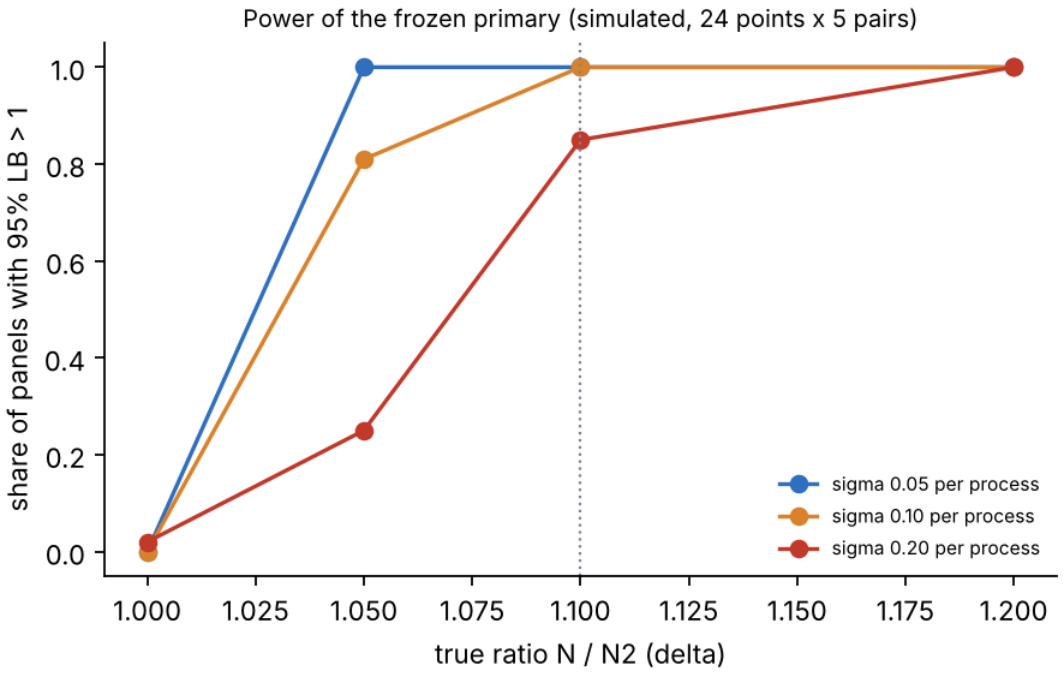


Figure 3: Power of the frozen primary (simulated, new seed). Vector: figures/power.svg.

Power (`analysis/power.json`, re-run with the new seed). At a true ratio of 1.10:

- power is 1.00 for $\sigma \leq 0.10$ and 0.85 at $\sigma = 0.20$ (v1: 0.87);
- the rejection rate with no effect is at most 0.02;
- the A/A window fails in 0% of panels at $\sigma \leq 0.10$ and 12% at $\sigma = 0.20$.

v3's A/A was 1.0003, which points to the low σ. The expected effect is far larger: about 6.5 if the port cost nothing (the native solve was 15.4% of N's cold CPU in v3).

Local checks before freezing (cloud VM, no Sage; not timing evidence)

check	result
`cargo test --release --locked` (crate; local rustc 1.97.0, CI pins 1.99.0)	15 of 15
`tests/test_verify_receipt.py` (planted corruptions, crash pre-check)	22 of 22
`tests/test_pyrand.py`, `test_n2x_compare.py`, `test_archived_identity.py` (all 20 v3 points against the archived Sage B rows)	pass
`tests/test_option_path.py` (with v1's sources present)	6 of 6
dry run of the smoke part (stand-in `sage` that now executes the real option path)	every gate except `tier1_counters_recorded` (no `/proc/schedstat` here)
dry run with v1's `arm_n2x.sage` swapped in (negative control)	stops before the gates, exit 7, both N2X crashes named
`option_check.py` under an emulating `sage` shim	passes on v2 (5 launches); fails on v1's arm

Launch

The run has not been launched yet. This section records it once it is.

Files

- **Code.**
  - `native/`: the crate, unchanged from v1, with `src/bin/tau6dlp.rs` as arm N2;
  - `arm_n2x.sage` (the planted arm, **fixed**) and `n2x_compare.py`;
  - `option_check.py` (**new**: the Sage option-path gate);
  - `run_schedule.py`, `verify_receipt.py` (**crash pre-check and gate**) and `stats.py`.
- **Frozen inputs.**
  - `make_protocol.py` → `protocol.json`;
  - `make_panel.py` → `inputs/panel.json` (with `inputs/excluded/`);
  - `make_schedule.py` → `schedule.json`;
  - `power_check.py` → `analysis/power.json`.
- **Baseline, unchanged.** `../volcano-n19-native-tau6-v3-20261020/arm.sage` and its `native/`.
- **Checks.**
  - `tests/`, including `sage_emulation.py`, `namespace_audit.py` and `test_option_path.py`;
  - `smoke/`, with the dry runs, the negative control and the emulated option check.
- **Diagnosis.** `diagnosis/profile_n.sage` (**fixed**) and `diagnosis/startup_probe.py`.
- **Report.** `make_figures.py`, `figures/`, `README.pdf` and `report_header.tex`.